The Pillar Message Format: One Sealed, Content-Addressed Envelope for Every Byte Pillar Persists or Transmits

pillar engineering (machine-checked design note)

2026-09-09

Abstract

Pillar today carries what is fundamentally one thing—a signed, content-addressed record—in three unrelated byte formats: streamdb ops (hand-rolled length-prefixed `SignedSegment`, persisted to an embedded IPFS node), observability signals (an in-memory timeseries struct, never persisted or replicated), and libp2p control traffic (per-protocol CBOR, encrypted only by the transport's Noise session). Three formats mean three serializers, three version stories, three security postures—and observability gets none of the durability, content-addressing, or replication streamdb already has. We unify them behind a single record: the `PillarMessage`, a version-stamped envelope whose payload is AEAD-sealed to the owning *cell* and addressed by the SHA2-256 multihash of its canonical CBOR encoding. A body's seal is chosen by *how many parties must unseal it*: the transport-frame seal (a datagram fans out to many ingest nodes) and the `PillarMessage` content seal (a message reaches every cell node) are both *convergent*—the key is portable and the nonce is derived (HKDF) from the cell key and the content address of the plaintext, so identical bodies produce an identical `Cid` on every node and redundant sprayed datagrams and duplicate blocks dedupe, while distinct plaintexts never share a nonce (AEAD-safe). The *contents* of a direct-message Op reach one recipient, so they are instead *random*-sealed (unique per message, *not* content-deduped) and referenced by `Cid` from a cell-sealed wrapper; some control bodies ride unencrypted. streamdb ops, observability signals, and every control message become body variants of the one envelope, owned by a new shared crate `pillar-wire` onto which streamdb, observability, and net all depend *down*. On the wire the envelope rides a pillar-UDP datagram sealed under a portable, cell-minted session key (companion paper), or a QUIC/TCP connection under the transport's own TLS; the content seal is transport-agnostic and holds either way, so no transport ever sees plaintext. Per Pillar's method #1, this is the design of record and the implementation is TLA$^+$-gated: `OneRecordFormat`, `ContentAddressStable`, `DedupUnderEncryption`, `CellConfidential`, `NoNonceReuseAcrossDistinctPlaintext` (envelope), and `ConvergentCellDedup`, `RcptRandomDistinctByEntropy`, `RcptOpaqueToNonHolder` (body treatments) must be green under TLC before any Rust.

1 Motivation

The three formats Pillar uses today for one logical object—a signed, content-addressed record—are: **streamdb ops**, `SignedSegment{bytes,signer,signature,visibility}`, a hand-rolled length-prefixed wire form addressed by `Cid=content_address(to_wire())` and persisted through `ContentStore/IpfsBackend` onto Pillar's embedded IPFS node; **observability signals**, `Signal{id,kind,payload,labels,expiry}`, held only in an in-memory `TimeseriesStore`, never persisted, never on the wire; and **libp2p control traffic**, per-protocol CBOR request/response types plus gossipsub event-log messages, each its own struct, encrypted only by the libp2p Noise session at the transport layer.

Three formats mean three serializers, three version stories, three security postures, and—critically—observability signals get *none* of the durability, content-addressing, or replication that streamdb ops already have. The five signal kinds (metric/log/trace/profile/metadata) are already specified to ride "IPFS (durable signed segments/backfill) + streaming-DB tip … no parallel store, no second authority" (ROI Priority 3). This paper makes that literal by giving signals the *same* record type streamdb ops use.

Thesis. There is exactly one Pillar record on the wire and on disk: the `PillarMessage`, a version-stamped envelope carrying a sealed, content-addressed payload, of which streamdb ops, observability signals, and every control message are variants. It is (1) **content-addressed**—its `Cid` is the SHA2-256 multihash of its canonical encoding, identical on every node; (2) **sealed to the cell**—the payload is AEAD-encrypted under the cell group key, *convergently* so the `Cid` is stable across nodes despite encryption; (3) **version-stamped**—an independently incrementable surface version, per the ROI versioning spine; and (4) **the payload of a pillar-UDP datagram** (fallback QUIC, then TCP+TLS).

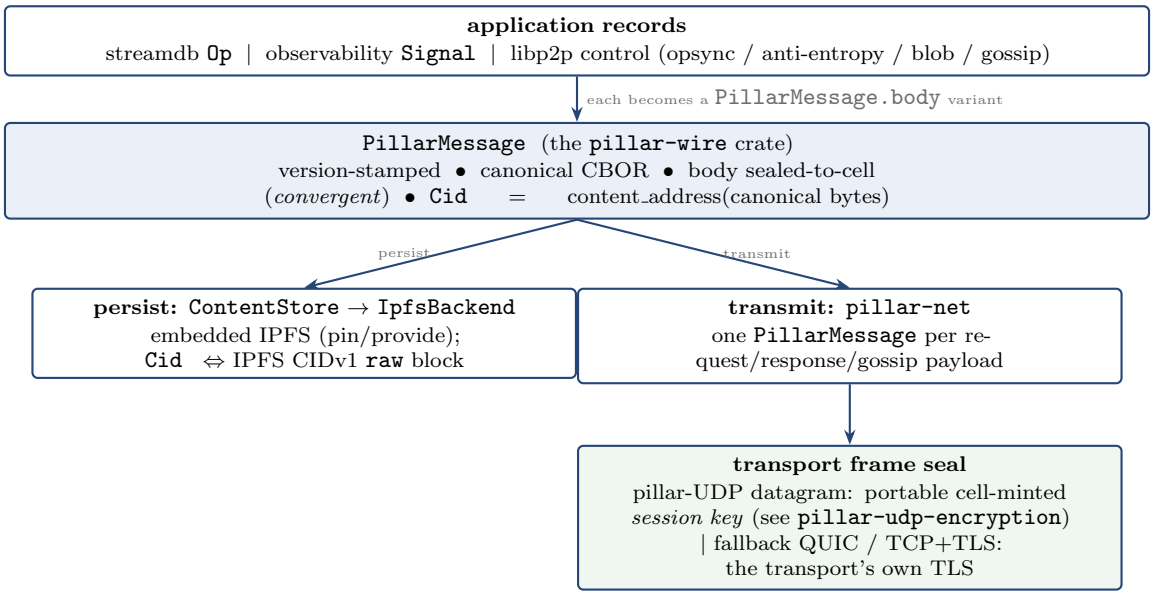

Figure 1: Layering. Every application record becomes the body of one `PillarMessage`; the envelope is content-addressed and cell-sealed once, then either persisted as an IPFS block or transmitted by `pillar-net`. The body seal travels *with* the record (an on-disk block is already sealed; a bitswap peer outside the cell cannot read it), so it is independent of, and composes with, the transport frame seal below—which is the pillar-UDP session key on pillar-UDP and the transport's native TLS on QUIC/TCP. A datagram to a cell therefore carries a cell-sealed body inside a session-key-sealed datagram: double-sealed, each layer independent.

2 Three seal treatments

A body's seal is decided by *how many parties must unseal it* (Table 1). The **transport frame seal** (a datagram fanned out to many ingest nodes) and the **content seal** on `PillarMessage.body` (a message reaching every cell node) each require *many* recipients to unseal, so both are *convergent*: a portable key and a deterministic nonce make identical content byte-identical, which is what lets redundant sprayed datagrams and duplicate blocks dedupe. The **application-content seal** on the *contents* of a direct-message Op reaches *one* recipient, so it may draw a *random* per-message nonce—the message is unique and canonical for itself, so same-text messages at different times stay distinct and are not content-deduped. On QUIC/TCP the transport's native TLS plays the frame-seal role; a control body may ride unencrypted.

	transport frame	**content seal**	**app-content**
layer	pillar-UDP datagram	`PillarMessage` body	Op contents
unseal	many (ingest nodes)	many (cell nodes)	**one** (recipient)
key	session key K_s	cell group key	recipient sealed-box
nonce	**convergent**	**convergent**	**random**
dedup	redundant datagrams	identical blocks	none (unique)

Table 1: The seal is chosen by recipient count. The two multi-node layers are convergent (so redundant/duplicate copies dedupe); the single-recipient application-content seal is random (so each direct message is a distinct record—exactly-once is an application-layer concern, not the wire format). A control body may be unencrypted. The content seal is transport-agnostic and travels with the record; the transport frame seal is pillar-UDP-specific (its portable session key is the subject of the companion paper `pillar-udp-encryption`). On QUIC/TCP the transport's TLS supplies the frame seal instead.

3 `pillar-wire`: the shared substrate crate

Both streamdb and observability **depend down** onto one shared crate rather than sideways onto each other; that crate is `pillar-wire`—the common home for the wire/persistence primitives currently trapped inside `pillar-streamdb` (named because the *same* bytes go on the wire and to disk; the deliberately broad "widely used substrate" crate preferred over a scatter of opaque micro-crates). It owns `PillarMessage` (the envelope enum + its canonical CBOR codec + `Cid` derivation), the content seal (`cell_seal_convergent`/`cell_open_convergent`, wrapping `pillar-crypto`), and—*moved down* from `pillar-streamdb`— `SignedSegment`, `Cid`, `HeadRecord`, `Visibility`, `ContentStore`, and the `IpfsBackend` trait. The dependency direction is strictly downward, no cy-

cles:

`pillar-streamdb`, `pillar-observability`, `pillar-net` → `pillar-wire` → `pillar-crypto`, `pillar-core`, `pillar-ipfs`.

`pillar-wire` must *not* depend on `pillar-net` (net uses the envelope, not the reverse) nor on the two crates that depend on it.

4 The envelope

The one record—persisted and transmitted—and the body it seals:

```
struct PillarMessage {
  version     :  SurfaceVersion
  signer      :  SigningPublicKey
  signature   :  Signature // over body_sealed
  visibility  :  Visibility // DHT reach
  cell        :  CellId // which group key
opens it
  body_sealed :  Ciphertext // per treatment
(§5)
}
enum Body { // what body_sealed decodes to
  StreamOp(StreamOpBody), // op-log; also
typed
      // UserMessage/NodeMessage/KeyOffer:
      // {recipient, sealed_cid} (§5b)
  Signal(SignalBody),   // one of 5 kinds
  Control(ControlBody),  // may be plaintext
(§5c)
}
```

Canonical encoding. The envelope and `Body` are encoded with **deterministic CBOR** (RFC 8949 §4.2.1: sorted map keys, definite lengths, shortest-int)—CBOR because `pillar-net` already uses `request_response::cbor`, and determinism because the `Cid` is a hash of these bytes, so two nodes MUST produce byte-identical encodings for one logical record or content-addressing breaks. This replaces streamdb's hand-rolled `to_wire`; `SignedSegment::to_wire/from_wire` become the `version=1` read-compatibility path (§7).

Content address. `Cid` = content_address(canonical_cbor(`PillarMessage`)), the existing `pillar-crypto` SHA2-256 multihash—the same function streamdb and observability already share, and the same multihash inside an IPFS CIDv1 `raw` block. Signing is over `body_sealed` (the ciphertext), so signature verification needs no cell key, decryption needs the cell key, and both are independent of the `Cid`.

5 Seal treatments

(a) Convergent, where many must unseal. A `PillarMessage` reaches every node in its cell and they must *all* decrypt, so the content seal is *convergent*: *the recipient is the cell; any cell node decrypts; the `Cid` is the same across nodes; we deliver BOTH dedup AND encryption.* Naïvely this is impossible—`pillar-crypto::seal_symmetric` draws a *random* nonce, so the same signal sealed on two nodes yields different ciphertext, a different `Cid`, and no dedup. We therefore add a *convergent* seal:

$$\begin{aligned} nonce &= \mathrm{HKDF}\big(\mathrm{key}{=}cell\_group\_key,\\ &\qquad \mathrm{info}{=}\text{“}\ldots/\text{convergent-nonce/v1”}\,\|\,domain,\\ &\qquad \mathrm{ikm}{=}\mathrm{content\_address}(plaintext)\big)[..n]\\ ct &= \mathrm{AEAD\_seal}(cell\_group\_key, nonce, plaintext,\\ &\qquad \mathrm{aad}{=}envelope\_header) \end{aligned}$$

Its properties: **deterministic**—the nonce is a pure function of (cell key, plaintext), so identical body + identical cell key ⇒ identical ciphertext ⇒ identical `Cid` on every node (the streamdb CRDT idempotence guarantee, now for sealed content); **confidential**—recovered only with the cell group key, itself sealed to each member by the existing `distribute_group_key` (X25519 sealed-box), so a non-member or bitswap peer holds only ciphertext; **nonce-safe**—reuse happens *only* for identical plaintext (which produces identical ciphertext anyway—the intended dedup), never across distinct plaintexts, because the nonce binds content_address(*plaintext*); and an **equality leak**, accepted and documented—convergent encryption reveals that two sealed records are byte-identical (that is *how* dedup works), a property scoped to the cell confidentiality boundary, with the `domain` input separating record classes so equality never leaks across classes. This convergent seal is what a StreamOp, an obs signal, and a direct message's cell *wrapper* get; it is exactly what a random nonce would break, and why the two multi-node layers (content seal and transport frame) are convergent (proven in `PillarMessageFormat.tla`).

(b) Random, where one recipient unseals. The *contents* of a direct-message Op—a typed `UserMessage/NodeMessage/KeyOffer`—are sealed to *one* recipient (X25519 sealed-box) and referenced by `Cid` from a cell-sealed wrapper. The cell sees the op and the reference but never the plaintext (it coordinates delivery without reading it). Because only one party unseals, the seal draws a *random* per-message nonce, and should: the message is unique and canonical for itself, so two same-text messages at different times are *distinct* records with distinct `Cids`—*not* content-deduped. The only dedup here is the byte-identity of truly redundant copies (one message sprayed to many nodes). *Exactly-once coordination is an application-layer concern (a CP-variant streamdb or other primitive), not the wire format.*

(c) Unencrypted, for one-shot non-addressed control. Some bodies—typically libp2p control messages—are sealed once by the transport and not content-addressed, so they ride as plaintext inside the envelope; any holder reads them. Treat-

ments (b) and (c) are proven in `PillarBodySeals.tla` (`RcptOpaqueToNonHolder`, `RcptRandomDistinctByEntropy`, `DedupByCid`, `PlainReadableWithoutKey`).

6 Transport

The envelope's on-wire encryption depends on the transport, and there are exactly two cases. **pillar-UDP** (a bare datagram substrate) supplies its own frame encryption keyed by a **portable, cell-minted session key**—established by a *cell-as-KDC* scheme distributed over streamdb, with the session key derived deterministically from the client's ephemeral key against the cell static key. It replaces the libp2p Noise upgrade; it is neither handshakeless-per-peer nor a per-pair Noise session—the key is portable across every cell node, so any ingress/relay can carry the session. The complete scheme (packet flow, key derivation, anonymous handling, revocation/GC, the forward-secrecy tradeoff) is its own design of record, the companion paper `pillar-udp-encryption`, TLA$^+$-gated by `specs/PillarUdpEncryption.tla`. **QUIC/TCP** (the fallbacks) bring their own **TLS 1.3**; the pillar-UDP scheme does not apply there. The selection order is pillar-UDP → QUIC → TCP+TLS, and because the body is *already* cell-sealed (§5), even the fallback transports never see plaintext application content. All libp2p control messages ride *inside* the envelope, exactly like ops and signals; pillar-UDP sits beneath IPFS in the stack. Removing the Noise upgrade is a deliberate, *breaking* change, safe only behind the versioning spine (§7).

7 Compatibility & versioning

Per the ROI versioning spine (independent per-surface stamps, $N-1$ window): the **envelope version** (`PillarMessage.version`) is `v1`= the legacy `SignedSegment` length-prefixed form (read-compat), `v2`= this canonical-CBOR convergent envelope; the **pillar-UDP protocol version** bumps for the Noise→session-key cutover; and the **sealed-artifact version** is the convergent-seal algorithm tag (its KDF carries a `"/v1" info` so a future scheme coexists). A stamped-but-unknown *future* version is rejected *distinctly* from a parse error, so a node one version ahead is refused cleanly, not silently mis-read. Migration is read-through: a `v1 Cid` stays valid, new writes are `v2`, and the content-addressed store lets both coexist by construction—no history rewrite.

8 What changes, concretely

New crate `pillar-wire` (envelope + codec + convergent seal + the primitives moved down from streamdb). `pillar-crypto` gains `cell_seal_convergent`/`cell_open_convergent` alongside the existing random-nonce `seal_symmetric`. `pillar-streamdb Op`/`OpLog` payloads become `StreamOp` bodies; persistence re-exports `pillar-wire`'s `ContentStore` (no CRDT/Merkle change). `pillar-observability Signal`s become `Signal` bodies, each persisted as a sealed, content-addressed IPFS block via `ContentStore`, with cross-node backfill on the same substrate as streamdb. `pillar-net` makes every payload a `PillarMessage`, drops the pillar-UDP Noise upgrade for the portable session key, and turns the per-protocol CBOR types into `ControlBody` variants.

9 Invariants (before any Rust)

The design is TLA$^+$-gated (method #1). It refines/extends the existing `versioning-compat-migration-spec` and `ObsIngestionSubstrate`; Table 2 lists the envelope/content and per-body-treatment obligations, all green under TLC in `specs/PillarMessageFormat.tla` (convergent envelope + version negotiation) and `specs/PillarBodySeals.tla` (the three seal treatments). The transport-frame obligations (portable session-key convergence, cell-signed session records, anonymous-as-unattested policy, replay-via-dedup, revoked-key erasure) are proven separately in `specs/PillarUdpEncryption.tla`.

10 Resolved design points

Anonymous-session transport crypto—*resolved*: anonymous clients use the *exact same* pillar-UDP session-key scheme; anonymity is a *policy* property, not a crypto scheme (an anonymous key is cryptographically equivalent to a user/node key and subject to the same WoT/RBAC checks; lacking attestations, the cell withholds sensitive data and privileged operations by default-deny). **Transport forward secrecy**—*accepted tradeoff*: the portable session key is derivable under the cell static key, so transport FS bounds to cell-key security plus erase-on-revoke—the same price the content seal already pays, with per-session ephemeral FS deliberately traded for cross-node portability. **Convergent-encryption equality leak (§5)**—accepted as the mechanism of dedup, scoped to the cell boundary, flagged so it is an explicit reviewed acceptance rather than an accident. The first two are detailed in `pillar-udp-encryption`.

Invariant	Guarantee
envelope + convergent content seal (`PillarMessageFormat.tla`)	
`OneRecordFormat`	every persisted/transmitted byte is a `PillarMessage`
`ContentAddress-Stable`	`Cid` is a deterministic function of the logical record, equal across nodes despite sealing
`DedupUnder-Encryption`	identical sealed records collapse to one `Cid`/one block
`CellConfidential`	only a cell-key holder recovers `Body`; a non-member/bitswap peer gets ciphertext only
`NoNonceReuseAcross-DistinctPlaintext`	convergent content nonces collide only for identical plaintext
`Negotiation-Refuses-Incompatible`, `RollingCoexistence`	envelope, pillar-UDP, and seal versions bump independently; a mixed-version (incl. Noise-era) swarm never mis-frames or partitions
per-body seal treatments (`PillarBodySeals.tla`)	
`Convergent-CellDedup`	a cell-sealed body's `Cid` is a pure function of (cell, content); identical content collapses, distinct never collides
`RcptRandom-DistinctByEntropy`	a random single-recipient seal binds per-message entropy; same-content direct messages at different times stay distinct (not deduped)
`RcptOpaque-ToNonHolder`	a recipient-sealed Op body opens only for the recipient's key-holder; the cell holds only the referenced `Cid`
`DedupByCid`, `Plain-ReadableWithoutKey`	redundant/duplicate copies collapse by `Cid`; an unencrypted control body is readable with no key

Table 2: TLA$^+$ obligations across the two envelope specs: the convergent envelope + version negotiation (`PillarMessageFormat.tla`) and the three per-body seal treatments (`PillarBodySeals.tla`).

References

[1] C. Bormann and P. Hoffman, *Concise Binary Object Representation (CBOR)*, RFC 8949, 2020.

[2] J. Benet, *IPFS—Content Addressed, Versioned, P2P File System*, arXiv:1407.3561, 2014.

[3] H. Krawczyk, *Cryptographic Extraction and Key Derivation: The HKDF Scheme*, CRYPTO, 2010.

[4] L. Lamport, *Specifying Systems*, Addison-Wesley, 2002.